

Proton qTMD and PDF Measurement

Physics and Contraction Note

PyQUDA Measurement Collaboration

July 20, 2026

Contents

1	Physics Overview	2
2	Proton Interpolating Field	3
3	Proton Two-Point Function	3
3.1	Definition	3
3.2	Wick Contraction – Complete Derivation	4
3.3	Diagrammatic Interpretation	5
4	Optimized Contraction Code	6
4.1	Term 1 Decomposition	6
4.2	Term 2 Decomposition	6
4.3	Final Summation	7
5	Gamma Matrix Conventions	7
5.1	Sink Bilinears	7
5.2	Interpolator Matrices	8
6	Connected Three-Point qTMD Function	8
6.1	Definition	8
6.2	Data Layout	9
7	Fixed-Sink Sequential Sources	9
7.1	Up-Quark Insertion (Flavor 1)	9
7.2	Down-Quark Insertion (Flavor 2)	10
7.3	Spin Projections	10
8	CG qTMD Operator	11
8.1	Incremental Shift Strategy	11
9	PDF Operators	11
9.1	CG PDF	11
9.2	GI PDF	12
9.3	Comparison	12
10	Wilson Line Index Construction	12
10.1	CG qTMD Wilson Indices	12
10.2	PDF Wilson Indices	13

11 Code Implementation	13
11.1 Constructor Parameters	13
11.2 Method Summary	13
12 Array and Output Shape Convention	14
12.1 Internal Array Shapes	14
12.2 HDF5 Output Layout	14
12.2.1 Two-Point Function	14
12.2.2 Three-Point qTMD/PDF	14
13 Key Differences from Pion qTMD	15
14 Limitations	15
15 Connected GI qTMD Update	15
16 Application Workflow Summary	16

1 Physics Overview

Proton transverse momentum dependent (TMD) distributions encode the three-dimensional momentum structure of the nucleon in QCD. Unlike the pion, the proton is a baryon composed of three valence quarks, which introduces two key complications relative to meson TMD measurements:

1. The proton interpolating field requires a color-antisymmetric contraction via the Levi-Civita tensor ε_{abc} , producing a baryon two-point function with two distinct contraction terms (direct and exchange).
2. Sequential sources are flavor-dependent: the up-quark insertion involves four epsilon-contracted terms, while the down-quark insertion involves two.

The leading-twist TMDs accessible from this measurement include:

- $f_1(x, k_\perp^2)$ – unpolarized quark distribution,
- $g_{1L}(x, k_\perp^2)$ – helicity distribution,
- $h_1(x, k_\perp^2)$ – transversity distribution,

where x is the longitudinal momentum fraction and $k_\perp$ is the intrinsic transverse momentum. The Fourier-conjugate of the TMD in impact-parameter space $b_\perp$ is the quasi-TMD (qTMD) wave function, which is the primary object computed in this lattice QCD measurement.

This document describes in full technical detail: (i) the Wick contraction of the proton two-point function, (ii) the sequential-source construction for the connected three-point qTMD function, (iii) the Wilson-line index generation for CG and GI-displaced propagators, (iv) the einsum decomposition strategy used in the PyQUDA GPU implementation, and (v) the HDF5 output conventions. The target audience is lattice QCD practitioners who need to understand every contraction index.

2 Proton Interpolating Field

The proton interpolating operator used throughout this measurement is the standard color-singlet three-quark field (suppressing the time argument for brevity):

$$\chi_\alpha(x) = \varepsilon_{abc} [u_a^T(x) C \Gamma_{\text{interp}} d_b(x)] u_{c,\alpha}(x), \quad (1)$$

where:

- $\alpha \in \{0, 1, 2, 3\}$ is the free Dirac index of the proton,
- $a, b, c \in \{0, 1, 2\}$ are fundamental color indices,
- ε_{abc} is the totally antisymmetric Levi-Civita tensor with $\varepsilon_{012} = +1$,
- $C = i\gamma_2\gamma_4$ is the charge-conjugation matrix in the Euclidean Dirac basis,
- Γ_{interp} selects the spin structure of the diquark.

Three interpolator choices are supported, denoted by string tags:

Tag	Matrix	Expression
"5"	$C\gamma_5$	$C\gamma_5$
"T5"	$C\gamma_t\gamma_5$	$C\gamma_t\gamma_5$
"Z5"	$C\gamma_z\gamma_5$	$C\gamma_z\gamma_5$

In the PyQUDA code (using the QUDA bitmask gamma basis), these are constructed as:

$$C\gamma_5 = (i\gamma_2\gamma_8) \cdot \gamma_{15}, \quad (2)$$

$$C\gamma_t\gamma_5 = (i\gamma_2\gamma_8) \cdot \gamma_7, \quad (3)$$

$$C\gamma_z\gamma_5 = (i\gamma_2\gamma_8) \cdot \gamma_{11}, \quad (4)$$

where the QUDA basis uses $\gamma_8 = \gamma_t$, $\gamma_7 = \gamma_t\gamma_5$, $\gamma_{15} = \gamma_5$, and $\gamma_{11} = \gamma_z\gamma_5$. The factor $i\gamma_2\gamma_8$ is the charge conjugation matrix in this basis.

The conjugate field is:

$$\bar{\chi}_\alpha(x) = \varepsilon_{def} \bar{u}_{d,\alpha}(x) [\bar{d}_e(x) \Gamma_{\text{interp}}^\dagger C^\dagger \bar{u}_f^T(x)], \quad (5)$$

where $\Gamma_{\text{interp}}^\dagger = \gamma_4 \Gamma_{\text{interp}}^\dagger \gamma_4$ in Euclidean space and $C^\dagger = C^{-1} = -C$.

3 Proton Two-Point Function

3.1 Definition

The momentum-projected proton two-point correlation function with a sink spin structure Γ_{sink} is:

$$C_2^{\Gamma_{\text{sink}}}(p, t) = \sum_x \exp(-ip \cdot (x - x_0)) \mathcal{P}_{\alpha\alpha'} \langle \chi_\alpha(x, t) \Gamma_{\text{sink}} \bar{\chi}_{\alpha'}(x_0, 0) \rangle, \quad (6)$$

where $\mathcal{P}_{\alpha\alpha'}$ is the spin projection matrix (e.g. unpolarized $P_+ = \frac{1}{4}(\gamma_0 + I)$ or polarized variants). The sum over x runs over the full spatial lattice at fixed time slice t .

3.2 Wick Contraction – Complete Derivation

Inserting the interpolating fields from Eqs. (1) and (5) into the expectation value, and expanding the quark fields:

$$\begin{aligned} \langle \chi_\alpha(x) \Gamma_{\text{sink}\alpha\alpha'} \bar{\chi}_{\alpha'}(x_0) \rangle &= \varepsilon_{abc} \varepsilon_{def} (C\Gamma_{\text{interp}})_{ij} (C\Gamma_{\text{interp}})_{kl} \Gamma_{\text{sink}mn}^{\alpha\alpha'} \\ &\times \langle u_a^i(x) d_b^j(x) u_c^\alpha(x) \bar{u}_d^m(x_0) \bar{d}_e^k(x_0) \bar{u}_f^n(x_0) \rangle, \end{aligned} \quad (7)$$

where the spinor indices are denoted by $i, j, k, l, m, n \in \{0, 1, 2, 3\}$ for the quark bilinears, with α, α' being the external proton spin indices. The color indices are $a, \dots, f \in \{0, 1, 2\}$.

Step 1: Identify the quark flavors. The interpolating field contains two up quarks and one down quark:

- u_a, u_c are up quarks (labeled by their Dirac indices and color positions),
- d_b is a down quark.

Similarly, the conjugate field contains $\bar{u}_d, \bar{u}_f$ (up) and $\bar{d}_e$ (down).

Step 2: Enumerate the Wick pairings. Wick's theorem instructs us to sum over all permutations of pairing each quark field with each anti-quark field. Because the up quarks u_a and u_c are identical fermions, there are two inequivalent ways to contract the up-quark operators:

Term	Pairing	Description
Term 1	$u_a \leftrightarrow \bar{u}_d, u_c \leftrightarrow \bar{u}_f$	Direct
Term 2	$u_a \leftrightarrow \bar{u}_f, u_c \leftrightarrow \bar{u}_d$	Exchange

The down quark always contracts as $d_b \leftrightarrow \bar{d}_e$, since there is exactly one down quark and one anti-down quark.

Step 3: Write each term with quark propagators. The quark propagator is defined as:

$$S_q(x, x_0)_{ab}^{i\alpha} \equiv \langle q_a^i(x) \bar{q}_b^\alpha(x_0) \rangle, \quad (8)$$

where i, α are spin indices and a, b are color indices. For isospin-symmetric computations ($m_u = m_d$), $S_u = S_d \equiv S$, so the same propagator is used for all three valence quarks.

Term 1 (Direct):

$$\begin{aligned} C_2^{(1)} &= \varepsilon_{abc} \varepsilon_{def} (C\Gamma_{\text{interp}})_{ij} (C\Gamma_{\text{interp}})_{kl} (\Gamma_{\text{sink}})_{mn} \\ &\times S(x, x_0)_{ad}^{ik} S(x, x_0)_{be}^{jl} S(x, x_0)_{cf}^{mn}, \end{aligned} \quad (9)$$

where the index assignments are:

- u_a (spin i , color a) $\rightarrow \bar{u}_d$ (spin k , color d): propagator $S(x, x_0)_{ad}^{ik}$,
- d_b (spin j , color b) $\rightarrow \bar{d}_e$ (spin l , color e): propagator $S(x, x_0)_{be}^{jl}$,
- u_c (spin $\alpha \equiv m$, color c) $\rightarrow \bar{u}_f$ (spin n , color f): propagator $S(x, x_0)_{cf}^{mn}$.

Term 2 (Exchange): The exchange term swaps the two up-quark assignments:

$$\begin{aligned} C_2^{(2)} &= \varepsilon_{abc} \varepsilon_{def} (C\Gamma_{\text{interp}})_{ij} (C\Gamma_{\text{interp}})_{kl} (\Gamma_{\text{sink}})_{mn} \\ &\times S(x, x_0)_{ad}^{ik} S(x, x_0)_{be}^{jn} S(x, x_0)_{cf}^{ml}, \end{aligned} \quad (10)$$

where now:

- u_a (spin i , color a) $\rightarrow \bar{u}_d$ (spin k , color d): same as Term 1,
- d_b (spin j , color b) $\rightarrow \bar{u}_f$ (spin n , color f): *exchanged* – d contracts to u ,
- u_c (spin m , color c) $\rightarrow \bar{d}_e$ (spin l , color e): *exchanged* – u contracts to d .

Note that in the exchange term, the flavor labels u and d are interchanged on the propagators (a down quark connects to the final $\bar{u}_f$, and an up quark connects to the final $\bar{d}_e$). Under isospin symmetry, all propagators are identical, so the flavor labels serve only to track the index routing.

Step 4: The sign convention. The fully antisymmetric contraction of two ε tensors with Fermion-exchange antisymmetry produces an overall minus sign:

$$C_2^{\Gamma_{\text{sink}}}(p, t) = -\left(C_2^{(1)} + C_2^{(2)}\right) \times \sum_x e^{-ip \cdot (x-x_0)} \mathcal{P}_{\alpha\alpha'}, \quad (11)$$

where $C_2^{(1)}$ and $C_2^{(2)}$ are defined by Eqs. (9) and (10) respectively. The negative sign originates from the fermion anticommutation required to bring the quark operators into canonical contraction order. After the Wick expansion, each term acquires a sign $(-1)^{n_P}$ where n_P is the parity of the permutation mapping the original operator ordering to the contracted pairing. Since each contraction is a product of three Wick pairs, the permutation that aligns the color-epsilon structure with the contracted propagator pairs yields $n_P = 1$ for both terms.

Step 5: Compact tensor notation. The two-point function can be written compactly in a factorized diagrammatic form. Omitting the external momentum phase and spin projection for clarity, and using position-space shorthand:

$$\text{Term 1: } \varepsilon_{abc} \varepsilon_{def} (C\Gamma_{\text{interp}})_{ij} (C\Gamma_{\text{interp}})_{kl} (\Gamma_{\text{sink}})_{mn} S_{ad}^{ik} S_{be}^{jl} S_{cf}^{mn}, \quad (12)$$

$$\text{Term 2: } \varepsilon_{abc} \varepsilon_{def} (C\Gamma_{\text{interp}})_{ij} (C\Gamma_{\text{interp}})_{kl} (\Gamma_{\text{sink}})_{mn} S_{ad}^{ik} S_{be}^{jn} S_{cf}^{ml}. \quad (13)$$

The key difference between the two terms is purely in the index routing of the third propagator and the exchange of indices (l, n) on the second and third propagators.

3.3 Diagrammatic Interpretation

Term 1 corresponds to the “direct” contraction:

$$\begin{array}{c} \text{u_a} \text{ -- } [C\Gamma] \text{ -- } \text{d_b} \quad \text{u_c} \text{ -- } [\Gamma_{\text{sink}}] \text{ -- } \text{ubar_f} \\ \text{with } \varepsilon_{abc} \text{ at source and } \varepsilon_{def} \text{ at sink.} \end{array}$$

Term 2 corresponds to the “exchange” contraction where the two identical up quarks are crossed:

$$\text{u_a} \text{ -- } [C\Gamma] \text{ -- } \text{d_b} \quad (\text{u and d sink legs exchanged}).$$

There are exactly two terms because the up quarks are identical fermions but are distinguished by their role in the interpolating field: one forms the scalar diquark with the down quark (via $C\Gamma_{\text{interp}}$), the other carries the open spin index. The two contractions differ in which up quark at the source connects to which up anti-quark at the sink.

4 Optimized Contraction Code

The function `contract_2pt_TMD` in `proton_qTMD_pyquda.py` implements the two-point contraction on GPU accelerators. The full 8-index tensor contraction (over $a, b, c, d, e, f, i, j, k, l, m, n$) is too large for a single einsum call (the intermediate tensors would exceed GPU memory). The code therefore splits the contraction into a sequence of smaller 2-tensor and 3-tensor einsum operations, carefully managing intermediate memory through explicit deletion.

The propagator data is shaped as $[\omega, t, z, y, x, \text{spin}_a, \text{spin}_b, \text{color}_a, \text{color}_b]$ where ω is the even-odd parity index. The momentum phases array is indexed as $[p, \omega, t, z, y, x]$. The 16 gamma matrices for the sink insertion form an array $[g, t_4, m, n]$ where $g \in \{0, \dots, 15\}$ labels the bilinear.

4.1 Term 1 Decomposition

The full Term 1 einsum would be:

$$\varepsilon_{abc} \varepsilon_{def} (CT_{\text{interp}})_{ij} (CT_{\text{interp}})_{kl} (P_{2\text{pt}})_{gt_4mn} S_{ad}^{ik} S_{be}^{jl} S_{cf}^{mn} \Phi_p \rightarrow [g, p, t], \quad (14)$$

where Φ_p is the momentum phase and $P_{2\text{pt}}$ holds the 16 sink gammas.

The decomposition proceeds in three stages:

Stage 1 — Sink block (split). The sink block contracts ε_{abc} with the first propagator and $(CT_{\text{interp}})_{ij}$ with the second propagator, then combines on indices i and b :

$$\text{t1_s1}: \quad \varepsilon_{abc} \times S_{ad}^{ik} \rightarrow [\omega, t, z, y, x, i, k, b, c, d], \quad (15)$$

$$\text{t1_s2}: \quad (CT_{\text{interp}})_{ij} \times S_{be}^{jl} \rightarrow [\omega, t, z, y, x, i, l, b, e], \quad (16)$$

$$\text{term1_sink}: \quad \text{t1_s1} \times \text{t1_s2} \rightarrow [\omega, t, z, y, x, k, l, c, d, e] \quad (\text{contract } i, b). \quad (17)$$

After this stage, the tensors `t1_s1` and `t1_s2` are deleted.

Stage 2 — P3 block. The sink gamma array is contracted with the third propagator:

$$\text{term1_p3}: \quad (P_{2\text{pt}})_{gmn} \times S_{cf}^{mn} \rightarrow [g, \omega, t, z, y, x, c, f]. \quad (18)$$

Stage 3 — Final assembly (split). The sink block and P3 block are combined and multiplied by the momentum phases:

$$\text{t1_f1}: \quad \varepsilon_{def} \times \text{term1_sink} \rightarrow [\omega, t, z, y, x, k, l, c, f] \quad (\text{contract } d, e), \quad (19)$$

$$\text{t1_f2}: \quad (CT_{\text{interp}})_{kl} \times \text{t1_f1} \rightarrow [\omega, t, z, y, x, c, f] \quad (\text{contract } k, l), \quad (20)$$

$$\text{t1_f3}: \quad \text{t1_f2} \times \text{term1_p3} \rightarrow [g, \omega, t, z, y, x] \quad (\text{contract } c, f), \quad (21)$$

$$\text{term1}: \quad \Phi_p \times \text{t1_f3} \rightarrow [g, p, t]. \quad (22)$$

Each intermediate tensor is deleted immediately after use.

4.2 Term 2 Decomposition

Term 2 has a different index routing (S_{be}^{jn} instead of S_{be}^{jl} and S_{cf}^{ml} instead of S_{cf}^{mn}), so the decomposition differs:

Stage 1 — Sink block.

$$\text{t2_s1}: \quad \varepsilon_{abc} \times S_{ad}^{ik} \rightarrow [\omega, t, z, y, x, i, k, b, c, d], \quad (23)$$

$$\text{t2_s2}: \quad (CT_{\text{interp}})_{ij} \times S_{be}^{jn} \rightarrow [\omega, t, z, y, x, i, n, b, e] \quad (\text{note: index } n \text{ not } l), \quad (24)$$

$$\text{term2_sink}: \quad \text{t2_s1} \times \text{t2_s2} \rightarrow [\omega, t, z, y, x, k, n, c, d, e] \quad (\text{contract } i, b). \quad (25)$$

Stage 2 — P3 block. The sink gamma contracts with the third propagator, keeping the exchange index l :

$$\text{term2_p3} : (P_{2\text{pt}})_{gmn} \times S_{cf}^{ml} \rightarrow [g, \omega, t, z, y, x, n, l, c, f]. \quad (26)$$

Note that m is contracted between $P_{2\text{pt}}$ and the propagator, while n and l remain open for the exchange contraction.

Stage 3 — Final assembly.

$$\text{t2_f1} : \varepsilon_{def} \times \text{term2_sink} \rightarrow [\omega, t, z, y, x, k, n, c, f] \quad (\text{contract } d, e), \quad (27)$$

$$\text{t2_f2} : \text{t2_f1} \times \text{term2_p3} \rightarrow [g, \omega, t, z, y, x, k, l] \quad (\text{contract } n, c, f), \quad (28)$$

$$\text{t2_f3} : (CT_{\text{interp}})_{kl} \times \text{t2_f2} \rightarrow [g, \omega, t, z, y, x] \quad (\text{contract } k, l), \quad (29)$$

$$\text{term2} : \Phi_p \times \text{t2_f3} \rightarrow [g, p, t]. \quad (30)$$

4.3 Final Summation

The complete two-point function is:

$$C_2[g, p, t] = -(\text{term1} + \text{term2}). \quad (31)$$

After the contraction, the result is MPI-gathered and saved as HDF5 (rank 0 only). The saved data is time-rolled by the negative of the source time to align $t = 0$ at the source.

5 Gamma Matrix Conventions

5.1 Sink Bilinears

The dense qTMD/PDF dataset `corr[wilson,momentum,gamma,time]` stores the raw PyQUDA bit-mask Gamma matrices actually used in the contraction. The HDF5 attribute `gamma_basis_schema` has value

`pyquda_bitmask16_with_physics_transform_v1.`

The exact IDs, matrices, and raw-to-physical transform are stored in `gamma_pyquda_ids`, `gamma_matrices`, and `physical_from_pyquda`. The sixteen raw channels are ordered as:

Index	Label	Raw production matrix
0	5	γ_5
1	T	γ_t
2	T5	$-\gamma_t\gamma_5$
3	X	γ_x
4	X5	$\gamma_x\gamma_5$
5	Y	γ_y
6	Y5	$-\gamma_y\gamma_5$
7	Z	γ_z
8	Z5	$\gamma_z\gamma_5$
9	I	identity
10	SXT	$[\gamma_x, \gamma_t]/2 = \gamma_x\gamma_t$
11	SXY	$[\gamma_x, \gamma_y]/2 = \gamma_x\gamma_y$
12	SXZ	$[\gamma_x, \gamma_z]/2 = \gamma_x\gamma_z$
13	SYT	$[\gamma_y, \gamma_t]/2 = \gamma_y\gamma_t$
14	SYZ	$[\gamma_y, \gamma_z]/2 = \gamma_y\gamma_z$
15	SZT	$[\gamma_z, \gamma_t]/2 = \gamma_z\gamma_t$

In the QUDA bitmask gamma basis, these correspond to the gamma matrix ID array:

$$\text{pyquda_gammas_order} = [15, 8, 7, 1, 14, 2, 13, 4, 11, 0, 9, 3, 5, 10, 6, 12]. \quad (32)$$

The stored physics transform is

$$\Gamma_A^{\text{phys}} = \sum_B (\text{physical_from_pyquda})_{AB} \Gamma_B^{\text{raw}}. \quad (33)$$

For schema v1 only the Y5 and T5 diagonal entries are -1 , so the transformed axial labels uniformly mean $\gamma_\mu \gamma_5$. Raw tensor channels do not contain an additional factor of i ; multiply them by i only when using the Hermitian convention $i[\gamma_\mu, \gamma_\nu]/2$. The complete schema registry and a portable HDF5 conversion example are in `docs/EMT_gamma_and_raw_bilinears.md`.

5.2 Interpolator Matrices

The proton interpolator matrices CT_{interp} described in Section 2 are pre-computed and stored as 4×4 complex arrays. In the QUDA basis:

- $\text{Cg5} = (i\gamma_2\gamma_8) \cdot \gamma_{15} \leftrightarrow C\gamma_5$,
- $\text{CgT5} = (i\gamma_2\gamma_8) \cdot \gamma_7 \leftrightarrow C\gamma_t\gamma_5$,
- $\text{CgZ5} = (i\gamma_2\gamma_8) \cdot \gamma_{11} \leftrightarrow C\gamma_z\gamma_5$.

The choice of interpolator enters the two-point contraction as the $(CT_{\text{interp}})_{ij}$ matrix applied to the propagator indices i, j and k, l .

6 Connected Three-Point qTMD Function

6.1 Definition

The connected three-point qTMD function for a flavor insertion $f \in \{U, D\}$ is:

$$C_3^{\Gamma, f}(q, b, \tau; p_f, t_{\text{sep}}) = \sum_x e^{+iq \cdot (x - x_0)} \text{Tr}_{\text{spin, color}} \left[\text{Seq}_f(x; p_f, t_{\text{sep}}, \mathcal{P}) \right. \\ \left. \times \Gamma \times \mathcal{O}_b S_q(x, x_0) \right], \quad (34)$$

where:

- $\text{Seq}_f(x; p_f, t_{\text{sep}}, \mathcal{P})$ is the fixed-sink sequential propagator for flavor f , which already contains the baryon sink contraction, the final momentum projection p_f , the sink time t_{sep} , and the spin projection $\mathcal{P}$,
- Γ is the bilinear insertion operator (one of the 16 gammas),
- $\mathcal{O}_b$ is the nonlocal displacement operator shifting the forward propagator by impact parameter $b = (b_\perp, b_z)$,
- $q = p_f - p_i$ is the momentum transfer,
- τ labels the operator insertion time slice.

6.2 Data Layout

The sequential propagator after the final γ_5 conjugation has shape:

$$[\text{polarization}, \omega, t, z, y, x_{cb}, \text{spin}_a, \text{spin}_b, \text{color}_a, \text{color}_b]. \quad (35)$$

The forward propagator (with the nonlocal shift) has shape:

$$[\omega, t, z, y, x_{cb}, \text{spin}_a, \text{spin}_b, \text{color}_a, \text{color}_b]. \quad (36)$$

The three-point contraction einsum in the application is:

$$\text{Seq}[:, j, i, c, f] \times \text{Gamma}[g, i, m] \times \text{S_shifted}[:, m, j, f, c] \rightarrow [\text{pol}, g, \omega, t, z, y, x_{cb}], \quad (37)$$

where indices are: p =polarization, g =gamma, i =spin_src, m =spin_sink, j =spin_b, c =color_a, f =color_b. This is followed by contraction with the momentum phase Φ_q to project onto momentum transfer q .

7 Fixed-Sink Sequential Sources

The fixed-sink sequential sources for the proton are constructed by `create_bw_seq_pyquda` in `bw_seq_pyquda.py`. The construction follows these steps for each polarization $\mathcal{P}$ and flavor:

1. **Boosted smearing** at the sink: `prop` $\rightarrow$ `boosted_smearing(prop, w, boost_out)`.
2. **Baryon contraction**: depending on the flavor, either `up_quark_insertion_pyquda` or `down_quark_insertion_pyquda` contracts the propagator with epsilon tensors and the polarization matrix $\mathcal{P}$ to form the sequential source at the sink time slice.
3. **Time slicing**: the source is restricted to time slice $t_{\text{sink}} = (t_{\text{src}} + t_{\text{sep}}) \bmod L_t$ using `sequential12`.
4. **Momentum phase and γ_5** : the source is multiplied by $e^{ip_f \cdot (x-x_0)}$ and left-multiplied by γ_5 :

$$\text{seq_data} = \gamma_5 \cdot \Phi_{p_f} \cdot \text{seq_data}^\dagger. \quad (38)$$

5. **Inversion**: the source is smeared (boosted) and inverted to obtain the sequential propagator.
6. **Post-inversion γ_5** : the sequential propagator is conjugated and multiplied by γ_5 on the right:

$$\text{final} = \text{prop_inv}^\dagger \cdot \gamma_5. \quad (39)$$

Thus complex conjugation converts the source phase $e^{+ip_f \cdot (x-x_0)}$ into the effective fixed-sink projector $e^{-ip_f \cdot (x-x_0)}$.

7.1 Up-Quark Insertion (Flavor 1)

The up-quark insertion function `up_quark_insertion_pyquda` constructs the sequential source from two up-quark propagators Q_u , one down-quark propagator Q_d , the interpolator Γ , and the spin projection $\mathcal{P}$.

Let Q_u and Q_d be reshaped to `[vol, 4, 4, 3, 3]` with indices `[site, spinsink, spinsrc, colorsink, colorsrc]`.

Internally, the following intermediate combinations are precomputed:

$$G^T DG : (\Gamma^T)_{ij} (Q_d)_{jkab} (\Gamma)_{kl} \rightarrow [\text{vol}, i, l, a, b], \quad (40)$$

$$\mathcal{P} D_u : (\mathcal{P})_{ij} (Q_u)_{jkab} \rightarrow [\text{vol}, i, k, a, b], \quad (41)$$

$$D_u \mathcal{P} : (Q_u)_{jkab} (\mathcal{P})_{kl} \rightarrow [\text{vol}, j, l, a, b], \quad (42)$$

$$\text{Tr}[D_u \mathcal{P}] : (Q_u)_{kjab} (\mathcal{P})_{jk} \rightarrow [\text{vol}, a, b]. \quad (43)$$

The four epsilon-contracted terms are:

$$R_1 : \varepsilon_{abc} \varepsilon_{def} (G^T DG)_{mnbe} (Q_u)_{mnad} \rightarrow [\text{vol}, b, e, a, d] \xrightarrow{\times \varepsilon \varepsilon} [\text{vol}, c, f] \xrightarrow{\times \mathcal{P}} [\text{vol}, i, j, c, f], \quad (44)$$

$$R_2 : \varepsilon_{abc} \varepsilon_{def} (\text{Tr}[D_u \mathcal{P}])_{ad} (G^T DG)_{jibe} \rightarrow [\text{vol}, i, j, c, f], \quad (45)$$

$$R_3 : \varepsilon_{abc} \varepsilon_{def} (\mathcal{P} D_u)_{ikad} (G^T DG)_{jkbe} \rightarrow [\text{vol}, i, j, c, f], \quad (46)$$

$$R_4 : \varepsilon_{abc} \varepsilon_{def} (G^T DG)_{kiad} (D_u \mathcal{P})_{klbe} \rightarrow [\text{vol}, i, l, c, f] \xrightarrow{i \leftrightarrow l} [\text{vol}, i, j, c, f]. \quad (47)$$

The total sequential source is:

$$D_{\text{total}} = -(R_1 + R_2 + R_3 + R_4), \quad (48)$$

with a final spin swap to reorder the output as $[\text{vol}, \text{spin}_{\text{src}}, \text{spin}_{\text{sink}}, \text{color}_{\text{sink}}, \text{color}_{\text{src}}]$.

7.2 Down-Quark Insertion (Flavor 2)

The down-quark insertion `down_quark_insertion_pyquda` is simpler (two terms), since the down quark appears only once in the interpolating field. It takes a single propagator Q (isospin-symmetric), Γ , and $\mathcal{P}$.

The intermediate combinations are:

$$\mathcal{P} D : (\mathcal{P})_{ij} Q_{jiab} \rightarrow [\text{vol}, a, b], \quad (49)$$

$$G^T DG : (\Gamma^T)_{ij} Q_{jkab} (\Gamma)_{kl} \rightarrow [\text{vol}, i, l, a, b], \quad (50)$$

$$G^T D : (\Gamma^T)_{ij} Q_{jkab} \rightarrow [\text{vol}, i, k, a, b], \quad (51)$$

$$\mathcal{P} DG : (\mathcal{P})_{ij} Q_{jkab} (\Gamma)_{kl} \rightarrow [\text{vol}, i, l, a, b]. \quad (52)$$

The two terms are:

$$T_1 : \varepsilon_{abc} \varepsilon_{def} (\mathcal{P} D)_{fc} (G^T DG)_{uveb} \rightarrow [\text{vol}, u, v, a, d], \quad (53)$$

$$T_2 : \varepsilon_{abc} \varepsilon_{def} (G^T D)_{ujec} (\mathcal{P} DG)_{jkfb} \rightarrow [\text{vol}, u, k, a, d]. \quad (54)$$

The total is:

$$D_{\text{total}} = T_2 - T_1, \quad (55)$$

with the same final spin swap.

7.3 Spin Projections

The available spin projection matrices $\mathcal{P}$ are defined as:

$$P_+ = \frac{1}{4}(\gamma_0 + I), \quad (56)$$

$$S_z^+ = \gamma_0 - i\gamma_1\gamma_2, \quad S_z^- = \gamma_0 + i\gamma_1\gamma_2, \quad (57)$$

$$S_x^+ = \gamma_0 - i\gamma_2\gamma_4, \quad S_x^- = \gamma_0 + i\gamma_2\gamma_4. \quad (58)$$

The polarization projections stored in `PolProjections` are:

$$\text{PpUnpol} = P_+, \quad (59)$$

$$\text{PpSzp} = P_+ S_z^+, \quad (60)$$

$$\text{PpSzm} = P_+ S_z^-, \quad (61)$$

$$\text{PpSxp} = P_+ S_x^+, \quad (62)$$

$$\text{PpSxm} = P_+ S_x^-. \quad (63)$$

8 CG qTMD Operator

The Coulomb-gauge (CG) qTMD operator is a simple coordinate shift of the forward propagator, without explicit gauge links:

$$\mathcal{O}_b S_q(x, x_0) = S_q(x + b_\perp \cdot \hat{e}_\perp + b_z \cdot \hat{e}_z, x_0), \quad (64)$$

where $\hat{e}_\perp \in \{\hat{e}_x, \hat{e}_y\}$ selects the transverse direction (direction 0 or 1) and $\hat{e}_z$ is the longitudinal direction (direction 2 on the lattice). The displacement is parameterized by:

- $b_\perp \in \{0, 1, \dots, b_T\}$ – transverse displacement,
- $b_z \in \{-b_z, \dots, 0, \dots, b_z\}$ – longitudinal displacement.

8.1 Incremental Shift Strategy

Rather than shifting the propagator by the full vector $(b_\perp, b_z)$ from the origin for each Wilson-line index, the code maintains a running shifted propagator and applies only incremental shifts:

$$\text{prop_shifted} = \text{prop_prev}.\text{shift}(\Delta b_\perp, \text{dir}_\perp).\text{shift}(\Delta b_z, \text{dir}_z), \quad (65)$$

where $\Delta b_\perp = b_\perp^{\text{current}} - b_\perp^{\text{previous}}$ and $\Delta b_z = b_z^{\text{current}} - b_z^{\text{previous}}$. This keeps successive shifts small, minimizing communication overhead in the distributed lattice implementation.

The method `create_fw_prop_TMD_CG` implements this logic: it receives the current forward propagator, the current Wilson-line index W , and the previous index, then applies the needed relative shifts.

9 PDF Operators

The PDF operator is the $b_\perp = 0$ special case of the qTMD operator, involving only longitudinal displacements along the z -direction.

9.1 CG PDF

The Coulomb-gauge PDF uses ordinary lattice shifts with $b_\perp = 0$:

$$\mathcal{O}_{b_z}^{\text{CG}} S_q(x, x_0) = S_q(x + b_z \cdot \hat{e}_z, x_0). \quad (66)$$

This is implemented via the same `create_fw_prop_TMD_CG` method with $b_\perp = 0$ and $b_\perp^{\text{previous}} = 0$.

9.2 GI PDF

The gauge-invariant (GI) PDF uses covariant shifts along the z -direction, building an explicit straight Wilson line through successive gauge-covariant nearest-neighbor shifts:

$$\mathcal{O}_{b_z}^{\text{GI}} S_q(x, x_0) = U_z(x + (b_z - 1)\hat{e}_z) \cdots U_z(x + \hat{e}_z) U_z(x) S_q(x + b_z \cdot \hat{e}_z, x_0). \quad (67)$$

The implementation in `create_fw_prop_PDF_GI` uses `gauge.pure_gauge.covDev` to apply $\psi'(x) = U_\mu(x) \psi(x + \hat{\mu})$ for each nearest-neighbor step:

- direction 2: $+z$ (forward covariant derivative),
- direction 6: $-z$ (backward covariant derivative).

The code enforces nearest-neighbor steps only ($\Delta b_z \in \{-1, 0, 1\}$) and raises an error for larger jumps, protecting against accidentally skipping gauge links. The shift is applied spinor-by-spinor and color-by-color in a loop over the fermion field.

9.3 Comparison

Variant	Gauge links	Transverse	Code method
CG_TMD	None (CG)	x and y	<code>create_fw_prop_TMD_CG</code>
CG_PDF	None (CG)	none	<code>create_fw_prop_TMD_CG</code>
GLPDF	Covariant	none	<code>create_fw_prop_PDF_GI</code>

10 Wilson Line Index Construction

10.1 CG qTMD Wilson Indices

The CG qTMD requires two sets of Wilson-line indices, one for each transverse direction:

- `index_list_trans0`: transverse direction 0 (x),
- `index_list_trans1`: transverse direction 1 (y).

Each Wilson-line index is a 4-tuple $[b_T, b_z, \eta, \text{dir}]$ where:

- $b_T \in [0, b_T^{\text{max}}]$ – transverse displacement,
- b_z – longitudinal displacement,
- $\eta = 0$ (irrelevant for CG, kept for interface compatibility),
- $\text{dir} \in \{0, 1\}$ – transverse direction (x or y).

The generation algorithm:

1. For each $b_z \in [0, b_z^{\text{max}}]$, $b_T \in [0, b_T^{\text{max}}]$:
 - add $[b_T, b_z, 0, \text{dir}]$,
 - if $b_z \neq 0$, also add $[b_T, -b_z, 0, \text{dir}]$.
2. Reorder the index list to minimize adjacent differences in (b_T, b_z) . The reordering sorts by (b_T, b_z) first, then locally permutes to make each step as small as possible.

The final combined list is:

$$\text{W_index_list_CG} = \text{index_list_trans0} + \text{index_list_trans1}. \quad (68)$$

10.2 PDF Wilson Indices

The PDF Wilson-line index list contains only $b_T = 0$ entries:

1. For each $b_z \in [0, b_z^{\max}]$: add $[0, b_z, 0, 0]$.
2. For each $b_z \in [1, b_z^{\max}]$: add $[0, -b_z, 0, 0]$.

Since $b_T = 0$ for all entries, the transverse direction is always 0 (dummy). The positive and negative b_z values are separated to facilitate the incremental shift strategy: for GI-PDF, the gauge-covariant link construction requires resetting to the unshifted propagator when the sign of b_z changes (i.e., when $b_z = 0$ or $b_z = -1$ in the loop).

11 Code Implementation

The measurement is organized into a class `proton_TMD` in `proton_qTMD.pyquda.py`.

11.1 Constructor Parameters

Parameter	Description
<code>eta</code>	List of η values (irrelevant for CG)
<code>b_z</code>	Maximum longitudinal displacement
<code>b_T</code>	Maximum transverse displacement
<code>pilist</code>	Initial momenta p_i for 2pt
<code>width</code>	Gaussian smearing width
<code>boost_out</code>	C2 sink smearing

The application uses `boost_in` to smear the source before the forward inversion. The production measurement object stores `boost_out` and uses it for the sink side of the two-point contraction; the same `boost_out` is passed to the fixed-sink sequential-source construction. Thus the saved two-point function has the endpoint convention

$$C_2^{\text{out},\text{in}}(p, t) = \langle \chi_{S_{\text{out}}}(p, t) \bar{\chi}_{S_{\text{in}}}(0) \rangle. \quad (69)$$

The current canonical applications set `boost_in = boost_out = (0, 0, 0)`, so this correction does not change their standard zero-boost data. Manually produced older unequal-boost two-point data used `boost_in` at both endpoints and must be recomputed.

11.2 Method Summary

- `contract_2pt_TMD(latt_info, prop_f, phases, tag, interpolator)`: Full two-point contraction with all 16 gammas (Section 4).
- `create_TMD_Wilsonline_index_list_CG()`: Generate CG Wilson indices for both transverse directions (Section 10).
- `create_PDF_Wilsonline_index_list()`: Generate PDF Wilson indices ($b_{\perp} = 0$, Section 10).
- `create_fw_prop_TMD_CG(prop_f, W_index, WL_indices_previous)`: Incremental CG shift (Section 8).
- `create_fw_prop_PDF_GI(gauge, prop_f, W_index, WL_indices_previous)`: Gauge-covariant GI shift (Section 9).

12 Array and Output Shape Convention

12.1 Internal Array Shapes

Object	Shape
Propagator data	$[\omega, t, z, y, x_{cb}, s_a, s_b, c_a, c_b]$
Momentum phases	$[p, \omega, t, z, y, x]$ or $[p, \omega, t, z, y]$
Gamma array	$[g, m, n]$ ($16 \times 4 \times 4$)
Sequential propagator	$[\text{pol}, \omega, t, z, y, x_{cb}, s_a, s_b, c_a, c_b]$
C2 result	$[g, p, t]$
C3 result	$[\text{WL}, \text{pol}, q, g, t]$

where:

- ω : even-odd parity ($L_t \times L_z \times L_y \times L_x/2$),
- x_{cb} : checkerboarded x index,
- s_a, s_b : spin indices (0–3),
- c_a, c_b : color indices (0–2),
- g : gamma matrix index (0–15),
- p, q : momentum indices,
- pol : polarization index,
- WL : Wilson-line index.

12.2 HDF5 Output Layout

12.2.1 Two-Point Function

```
<tag>.h5
  /SS/<gamma>/PX..PY..PZ.. -> dataset of shape [Lt]
```

Each gamma channel stores all momenta as separate datasets. The time axis is rolled so that $t = 0$ corresponds to the source position.

12.2.2 Three-Point qTMD/PDF

```
<tag>.h5
  corr                [Wilson, momentum, gamma, time]
  gamma_list          [16]
  momentum_list       [N_momentum, 4]
  wilson_index_list    [N_Wilson, 4]
```

The Wilson-index columns are $[b_T, b_z, \eta, \text{dir}]$, where $\text{dir}=0,1$ denotes the X, Y transverse direction. PDF data uses the same schema with $b_T = 0$. Each file contains one operator, one flavor, and one polarization, while the complete 16-Gamma scan remains a dataset axis. The polarization is encoded in both the file tag and HDF5 attributes; it is not an additional **corr** axis.

The flavor-specific tags encode the flavor as part of the AMA field: "**CG.D.ex**" for down-quark CG insertion, "**CG.U.ex**" for up-quark CG insertion. The C2 is polarization independent and remains one shared file per source.

13 Key Differences from Pion qTMD

The proton qTMD measurement differs from the pion qTMD in several fundamental ways:

Aspect	Pion qTMD	Proton qTMD
Contraction	γ_5 -hermiticity	Baryon epsilon + 2 exchange terms
Quark lines	2 (one contraction term)	3 (two contraction terms)
Flavor dim.	None (single channel)	$[U, D]$ via 2 seq. sources
Polarization	None	Via spin projectors $\mathcal{P}$
Propagators	Separate fw/bw	Same prop reused (isospin)
Mom. params.	<code>pos_boost/neg_boost</code>	<code>boost_in/boost_out</code>
Seq. source	1 meson seq. source	2 baryon seq. sources
Output shape	$[\text{WL}, q, g, t]$	$[\text{WL}, \text{pol}, q, g, t]$

The isospin-symmetric approximation ($m_u = m_d$) means the same forward propagator is used for all three valence quark lines in the two-point function and for the sequential source construction. The up and down sequential sources differ only in their baryon contraction pattern, not in the input propagators.

14 Limitations

The current implementation has the following limitations:

1. **Connected diagrams only.** Disconnected insertions (where the operator couples to a sea-quark loop) are not included. These contribute to the singlet TMDs and may be substantial at low x .
2. **Isospin symmetry.** The code assumes $m_u = m_d$ and reuses the same forward propagator for all quark lines. Isospin-breaking effects from $m_u \neq m_d$ or from electromagnetic corrections are not captured.
3. **No flavor mixing or renormalization.** The bare lattice operators are not matched to the $\overline{\text{MS}}$ scheme. Renormalization factors and operator mixing (especially for the Wilson-line operators) must be applied externally.
4. **CG paths have no explicit gauge links.** The Coulomb-gauge qTMD operator is a simple coordinate shift. This is gauge-dependent unless the lattice is fixed to Coulomb gauge. Residual gauge-fixing uncertainties are not quantified.
5. **No disconnected TMD contributions.** The full proton TMD includes disconnected diagrams (operator insertion on a sea-quark loop connecting to the proton via gluons), which are not computed.
6. **Fixed sink only.** The sequential-source method fixes the sink time and momentum. To vary t_{sep} or p_f , separate inversions are required.

15 Connected GI qTMD Update

The connected proton qTMD code has a gauge-invariant staple path in addition to the Coulomb-gauge coordinate-shift qTMD and straight PDF operators. Both Perlmutter and Aurora call the single production implementation in `application/nucleon_TMD/shared_runner.py`; platform entry points only select backend and ensemble defaults.

The connected `GI_qTMD` operator uses the same Wilson index convention as the pion and disconnected qTMD code,

$$W = [b_T, b_z, \eta, d_\perp], \quad (70)$$

with $d_\perp = 0, 1$ for the transverse x or y direction. The fixed-length staple path is

$$x \rightarrow x + \left(\eta + \frac{b_z}{2} \right) \hat{z}, \quad (71)$$

$$\rightarrow x + \left(\eta + \frac{b_z}{2} \right) \hat{z} + b_T \hat{e}_{d_\perp}, \quad (72)$$

$$\rightarrow x + b_z \hat{z} + b_T \hat{e}_{d_\perp}. \quad (73)$$

The total staple length is therefore

$$L_{\text{staple}} = 2\eta + b_T, \quad (74)$$

which is independent of b_z . The allowed connected `GI_qTMD` indices satisfy

$$b_z \in 2\mathbb{Z}, \quad \eta \geq \frac{|b_z|}{2}, \quad b_T \geq 0. \quad (75)$$

The neutral `qtmd_operator_utils.py` module exposes this through:

- `create_gi_qtmd_wilsonline_index_lists(...)` generates the shared staple-index lists.
- `apply_gi_qtmd_staple_to_propagator(...)` applies a cached gauge-only staple transporter to each spin-color source column of the forward propagator.

The actual baryon contraction is unchanged: the shifted or covariantly transported forward line is contracted with the up- and down-insertion sequential propagators,

$$C_{3,g}^f(q, W, \tau) = \sum_{\mathbf{x}} e^{+i\mathbf{q} \cdot (\mathbf{x} - \mathbf{x}_0)} \text{Tr} [\text{Seq}_f(\mathbf{x}, \tau; p_f, t_{\text{sep}}, \mathcal{P}) \Gamma_g \mathcal{O}_W^{\text{GI}} S(\mathbf{x}, \tau; \mathbf{x}_0, 0)], \quad (76)$$

where $f = U, D$ labels the connected flavor insertion and $\mathcal{P}$ is the spin projector.

Output files use the tags

$$\text{GI_qTMD.U.ex}, \quad \text{GI_qTMD.D.ex}, \quad (77)$$

while the original CG files keep the tags `CG.U.ex` and `CG.D.ex`. Every operator/flux/polarization file uses the common dense layout based on $[b_T, b_z, \eta, d_\perp]$.

16 Application Workflow Summary

The production applications follow this workflow for each source position:

1. **Forward propagator:** Generate a point source at position (x_0, y_0, z_0, t_0) , apply boosted smearing with `boost_in`, and invert.
2. **Two-point contraction:** Call `contract_2pt_TMD` with the forward propagator and momentum phases, applying sink smearing with `boost_out`. This computes all 16 gamma channels and saves the result as HDF5.
3. **Sequential sources:** Build two sequential backward propagators via `create_bw_seq_pyquda`, using `boost_out` for the fixed sink:
 - Flavor 2 (down): `sequential_bw_prop_down`,

- Flavor 1 (up): `sequential_bw_prop-up`.
- 4. **TMD contractions (CG)**: Loop over Wilson-line indices for directions $+x$ and $+y$, incrementally shifting the forward propagator and contracting with both sequential propagators and the 16 gammas. Save one dense 16-Gamma file per polarization and flavor.
- 5. **TMD contractions (GI)**: In the new `application/nucleon.TMD` workflows, build cached fixed-length staple transporters, apply them to the forward propagator, and contract with the same sequential propagators. This path writes `GI_qTMD.U.ex` and `GI_qTMD.D.ex`.
- 6. **PDF contractions (GI)**: Loop over PDF Wilson-line indices, apply gauge-covariant z -shifts via `create_fw_prop_PDF_GI`, and contract similarly.
- 7. **PDF contractions (CG)**: Same as GI but using ordinary lattice shifts.

The momentum transfer q for the three-point functions is constructed from the `qext` parameter list: $q = (q_x, q_y, q_z, 0)$ with $q_z = 0$ for TMD and unrestricted for PDF. The initial momentum is $p_i = p_f - q$, and the two-point momenta are the negative of the `p_2pt` list.

On Perlmutter, the smoke-test entry point is

```
run_nucleon.TMD.sh --config_num CFG --mg-block 8.8.4.4. (78)
```

`--config_num` is mandatory and is never read from the environment. Unknown arguments fail before gauge I/O. The MG block is an explicit runtime choice; multiple levels use semicolon separation and `none` disables MG. MG, solver tolerance, and maximum iterations are not HDF5 provenance and do not enter the sample-log identity.

At startup rank zero reads the sample log using exact-line matching and broadcasts the completed-source set. Completed sources are skipped before source construction and inversion. A source is appended only after C2 and all products enabled in that invocation close successfully. The log is the only resume state and is intentionally independent of HDF5 location. It may therefore be reused only when operator switches, momentum/Wilson grids, smearing, interpolator, and sink separation are unchanged. Polarization is included in the sample-log identity, so completing one polarization does not suppress another. With all four operators enabled, one polarization produces one shared C2 file and eight three-point files.

The main environment toggles are `NUCLEON.TMD.RUN.CG.QTMD`, `NUCLEON.TMD.RUN.GI.QTMD`, and `NUCLEON.TMD.RUN.PDF`. The staple parameters are controlled by `NUCLEON.TMD.ETA`, `NUCLEON.TMD.BZ`, and `NUCLEON.TMD.BT`. For the fixed-length GI qTMD path, use even `NUCLEON.TMD.BZ` and choose `NUCLEON.TMD.ETA >= abs(NUCLEON.TMD.BZ)/2`.

Acknowledgments

This code was developed as part of the PyQUDA measurement framework for lattice QCD nucleon structure calculations. The contraction strategy and sequential-source construction build on established methods in the baryon spectroscopy literature.